

```
In [1]: import os
import cv2
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
In [2]: spiral_dir = r"C:\Users\Pritam\Spiral_HandPD"
meander_dir = r"C:\Users\Pritam\Meander_HandPD"

img_size = 128
```

```
In [3]: data = []
labels = []

def load_images(folder, label):
    for img_name in os.listdir(folder):
        if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):
            img_path = os.path.join(folder, img_name)
            img = cv2.imread(img_path)
            if img is None:
                continue
            img = cv2.resize(img, (img_size, img_size))
            img = img.astype("float32") / 255.0
            data.append(img)
            labels.append(label)

# Spiral
load_images(os.path.join(spiral_dir, 'healthy'), 0)
load_images(os.path.join(spiral_dir, 'parkinson'), 1)

# Meander
load_images(os.path.join(meander_dir, 'healthy'), 0)
load_images(os.path.join(meander_dir, 'parkinson'), 1)
```

```
In [4]: X = np.array(data, dtype=np.float32)
y_int = np.array(labels)
y = to_categorical(y_int, 2)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    shuffle=True
```

```
)

print("Train shape:", X_train.shape)
print("Test shape :", X_test.shape)

print("Train distribution:", np.unique(np.argmax(y_train, axis=1), return_counts=True))
print("Test distribution :", np.unique(np.argmax(y_test, axis=1), return_counts=True))
```

```
Train shape: (646, 128, 128, 3)
Test shape : (162, 128, 128, 3)
Train distribution: (array([0, 1]), array([175, 471]))
Test distribution : (array([0, 1]), array([ 41, 121]))
```

```
In [5]: # Softer than strict "balanced" to avoid over-correction
class_weights_dict = {
    0: 1.4, # Healthy (minority)
    1: 0.8 # Parkinson
}
print("Using class weights:", class_weights_dict)
```

```
Using class weights: {0: 1.4, 1: 0.8}
```

```
In [9]: cnn = Sequential([

    Conv2D(256, (3,3), padding="same", activation="relu", input_shape=(128,128,3)),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(32, (3,3), padding="same", activation="relu", name="feature_layer"),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(8, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(4, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
```

```
MaxPooling2D(2,2),

Conv2D(2, (3,3), padding="same", activation="relu"),
BatchNormalization(),
MaxPooling2D(2,2),

Flatten(),
Dropout(0.25),

Dense(128, activation='relu'),
BatchNormalization(),

Dense(2, activation='softmax')
])

cnn.compile(
    optimizer=Adam(learning_rate=5e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

cnn.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 128, 128, 256)	7,168
batch_normalization_5 (BatchNormalization)	(None, 128, 128, 256)	1,024
max_pooling2d_4 (MaxPooling2D)	(None, 64, 64, 256)	0
conv2d_4 (Conv2D)	(None, 64, 64, 128)	295,040
batch_normalization_6 (BatchNormalization)	(None, 64, 64, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 32, 32, 128)	0
conv2d_5 (Conv2D)	(None, 32, 32, 64)	73,792
batch_normalization_7 (BatchNormalization)	(None, 32, 32, 64)	256
max_pooling2d_6 (MaxPooling2D)	(None, 16, 16, 64)	0
feature_layer (Conv2D)	(None, 16, 16, 32)	18,464
batch_normalization_8 (BatchNormalization)	(None, 16, 16, 32)	128
max_pooling2d_7 (MaxPooling2D)	(None, 8, 8, 32)	0
conv2d_6 (Conv2D)	(None, 8, 8, 8)	2,312
batch_normalization_9 (BatchNormalization)	(None, 8, 8, 8)	32
max_pooling2d_8 (MaxPooling2D)	(None, 4, 4, 8)	0
conv2d_7 (Conv2D)	(None, 4, 4, 4)	292
batch_normalization_10 (BatchNormalization)	(None, 4, 4, 4)	16
max_pooling2d_9 (MaxPooling2D)	(None, 2, 2, 4)	0
conv2d_8 (Conv2D)	(None, 2, 2, 2)	74
batch_normalization_11 (BatchNormalization)	(None, 2, 2, 2)	8
max_pooling2d_10 (MaxPooling2D)	(None, 1, 1, 2)	0

flatten_1 (Flatten)	(None, 2)	0
dropout_1 (Dropout)	(None, 2)	0
dense_2 (Dense)	(None, 128)	384
batch_normalization_12 (BatchNormalization)	(None, 128)	512
dense_3 (Dense)	(None, 2)	258

Total params: 400,272 (1.53 MB)

Trainable params: 399,028 (1.52 MB)

Non-trainable params: 1,244 (4.86 KB)

```
In [10]: early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    min_lr=1e-6
)
```

```
In [14]: datagen = tf.keras.preprocessing.image.ImageDataGenerator(
    rotation_range=15,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='nearest'
)

datagen.fit(X_train)

cnn_history = cnn.fit(
    datagen.flow(X_train, y_train, batch_size=8),
    validation_data=(X_test, y_test),
    epochs=20,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.
self._warn_if_super_not_called()

Epoch 1/20
81/81 ————— **185s** 2s/step - accuracy: 0.5356 - loss: 0.6978 - val_accuracy: 0.6543 - val_loss: 0.6803 - learning_rate: 5.0000e-05
 Epoch 2/20
81/81 ————— **184s** 2s/step - accuracy: 0.6192 - loss: 0.6437 - val_accuracy: 0.7469 - val_loss: 0.6436 - learning_rate: 5.0000e-05
 Epoch 3/20
81/81 ————— **189s** 2s/step - accuracy: 0.6471 - loss: 0.6401 - val_accuracy: 0.6975 - val_loss: 0.6431 - learning_rate: 5.0000e-05
 Epoch 4/20
81/81 ————— **187s** 2s/step - accuracy: 0.6718 - loss: 0.6253 - val_accuracy: 0.6667 - val_loss: 0.6611 - learning_rate: 5.0000e-05
 Epoch 5/20
81/81 ————— **186s** 2s/step - accuracy: 0.6827 - loss: 0.6310 - val_accuracy: 0.7778 - val_loss: 0.6020 - learning_rate: 5.0000e-05
 Epoch 6/20
81/81 ————— **194s** 2s/step - accuracy: 0.6889 - loss: 0.6134 - val_accuracy: 0.8210 - val_loss: 0.5462 - learning_rate: 5.0000e-05
 Epoch 7/20
81/81 ————— **186s** 2s/step - accuracy: 0.7183 - loss: 0.6078 - val_accuracy: 0.7778 - val_loss: 0.5814 - learning_rate: 5.0000e-05
 Epoch 8/20
81/81 ————— **188s** 2s/step - accuracy: 0.7245 - loss: 0.5929 - val_accuracy: 0.7593 - val_loss: 0.5603 - learning_rate: 5.0000e-05
 Epoch 9/20
81/81 ————— **187s** 2s/step - accuracy: 0.7307 - loss: 0.5919 - val_accuracy: 0.7407 - val_loss: 0.5495 - learning_rate: 5.0000e-05
 Epoch 10/20
81/81 ————— **186s** 2s/step - accuracy: 0.7307 - loss: 0.5696 - val_accuracy: 0.7531 - val_loss: 0.5349 - learning_rate: 1.5000e-05
 Epoch 11/20
81/81 ————— **187s** 2s/step - accuracy: 0.7337 - loss: 0.5639 - val_accuracy: 0.7531 - val_loss: 0.5253 - learning_rate: 1.5000e-05
 Epoch 12/20
81/81 ————— **184s** 2s/step - accuracy: 0.7028 - loss: 0.5871 - val_accuracy: 0.7469 - val_loss: 0.5276 - learning_rate: 1.5000e-05
 Epoch 13/20
81/81 ————— **186s** 2s/step - accuracy: 0.7059 - loss: 0.5849 - val_accuracy: 0.7346 - val_loss: 0.5327 - learning_rate: 1.5000e-05
 Epoch 14/20
81/81 ————— **200s** 2s/step - accuracy: 0.7337 - loss: 0.5776 - val_accuracy: 0.7407 - val_loss: 0.5200 - learning_rate: 1.5000e-05
 Epoch 15/20
81/81 ————— **185s** 2s/step - accuracy: 0.7229 - loss: 0.5830 - val_accuracy: 0.7469 - val_loss: 0.5253 - learning_rate: 1.5000e-05
 Epoch 16/20
81/81 ————— **200s** 2s/step - accuracy: 0.7415 - loss: 0.5715 - val_accuracy: 0.7469 - val_loss: 0.5233 - learning_rate: 1.5000e-05
 Epoch 17/20
81/81 ————— **192s** 2s/step - accuracy: 0.7152 - loss: 0.5790 - val_accuracy: 0.7593 - val_loss: 0.5105 - learning_rate: 1.5000e-05
 Epoch 18/20
81/81 ————— **190s** 2s/step - accuracy: 0.7229 - loss: 0.5603 - val_accuracy: 0.7716 - val_loss: 0.4991 - learning_rate: 1.5000e-05
 Epoch 19/20
81/81 ————— **189s** 2s/step - accuracy: 0.7492 - loss: 0.5485 - val_accuracy: 0.7654 - val_loss: 0.5114 - learning_rate: 1.5000e-05
 Epoch 20/20
81/81 ————— **200s** 2s/step - accuracy: 0.7229 - loss: 0.5711 - val_accuracy: 0.7593 - val_loss: 0.5182 - learning_rate: 1.5000e-05

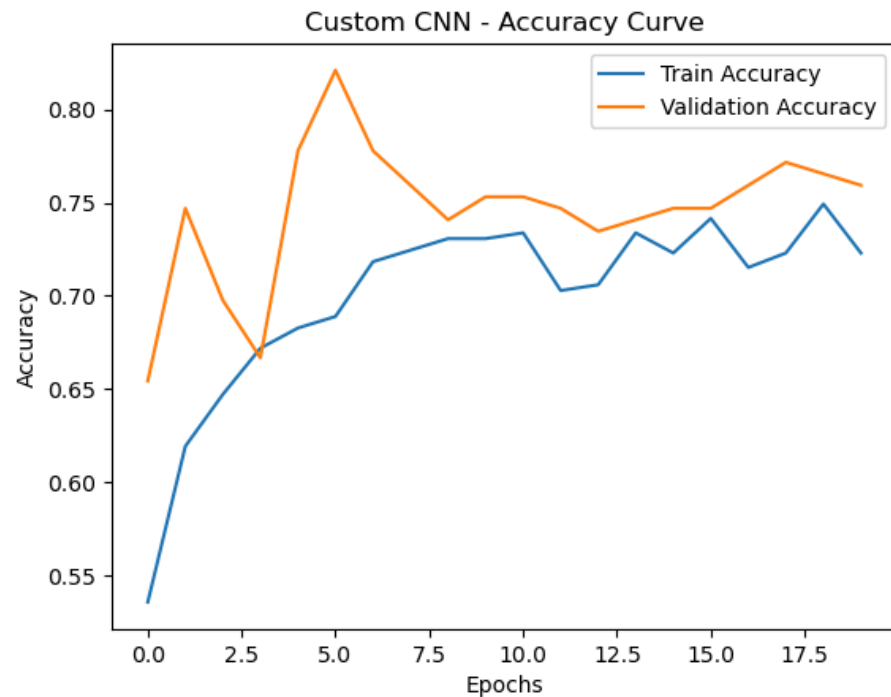
```
In [16]: cnn_loss, cnn_acc = cnn.evaluate(X_test, y_test, verbose=0)
print(" Final Custom CNN Accuracy:", cnn_acc)
print(" Final Custom CNN Loss:", cnn_loss)

preds = cnn.predict(X_test)
print("Predicted class distribution:",
      np.unique(np.argmax(preds, axis=1), return_counts=True))
print("True class distribution:",
      np.unique(y_test, return_counts=True))
```

Final Custom CNN Accuracy: 0.7716049551963806
 Final Custom CNN Loss: 0.49909722805023193
 6/6 ————— 8s 1s/step
 Predicted class distribution: (array([0, 1]), array([6, 156]))
 True class distribution: (array([0., 1.]), array([162, 162]))

In [17]: `import matplotlib.pyplot as plt`

```
plt.figure()
plt.plot(cnn_history.history['accuracy'], label='Train Accuracy')
plt.plot(cnn_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Custom CNN - Accuracy Curve')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



In [47]: `cnn.save("custom_cnn_parkinson.h5")`

```
print(" Custom CNN model saved as: custom_cnn_parkinson.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Custom CNN model saved as: custom_cnn_parkinson.h5

In [48]: *#Feature extraction*

```
In [49]: import tensorflow as tf
from tensorflow.keras.models import Model

input_tensor = tf.keras.Input(shape=(128,128,3))

x = input_tensor
for layer in cnn.layers:
    x = layer(x)

cnn = Model(inputs=input_tensor, outputs=x)

feature_extractor = Model(
    inputs=cnn.input,
    outputs=cnn.get_layer("conv2d_7").output
)

print(" Feature extractor successfully created from conv2d_7")
```



```

-----
TypeError                                Traceback (most recent call last)
Cell In[49], line 8
      6 x = input_tensor
      7 for layer in cnn.layers:
----> 8     x = layer(x)
     10 cnn = Model(inputs=input_tensor, outputs=x)
     12 feature_extractor = Model(
     13     inputs=cnn.input,
     14     outputs=cnn.get_layer("conv2d_7").output
     15 )

File ~\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\utils\traceback_utils.py:122, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    119 filtered_tb = _process_traceback_frames(e.__traceback__)
    120 # To get the full stack trace, call:
    121 # `keras.config.disable_traceback_filtering()`
--> 122 raise e.with_traceback(filtered_tb) from None
    123 finally:
    124     del filtered_tb

File ~\miniconda3\envs\data_analysis_env\lib\inspect.py:3186, in Signature.bind(self, *args, **kwargs)
    3181 def bind(self, /, *args, **kwargs):
    3182     """Get a BoundArguments object, that maps the passed `args`
    3183     and `kwargs` to the function's signature. Raises `TypeError`
    3184     if the passed arguments can not be bound.
    3185     """
-> 3186     return self._bind(args, kwargs)

File ~\miniconda3\envs\data_analysis_env\lib\inspect.py:3107, in Signature._bind(self, args, kwargs, partial)
    3105 param = next(parameters)
    3106 except StopIteration:
-> 3107     raise TypeError('too many positional arguments') from None
    3108 else:
    3109     if param.kind in (_VAR_KEYWORD, _KEYWORD_ONLY):
    3110         # Looks like we have no parameter for this positional
    3111         # argument

TypeError: too many positional arguments

```

```
In [25]: cnn.save("custom_cnn_parkinson.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [27]: from tensorflow.keras.models import load_model
```

```
cnn = load_model("custom_cnn_parkinson.h5")
print(" CNN loaded successfully")
```

WARNING:absl:No training configuration found in the save file, so the model was *not* compiled. Compile it manually.
CNN loaded successfully

```
In [28]: for layer in cnn.layers:
         print(layer.name)
```

```
input_layer_2
conv2d_3
batch_normalization_5
max_pooling2d_4
conv2d_4
batch_normalization_6
max_pooling2d_5
conv2d_5
batch_normalization_7
max_pooling2d_6
feature_layer
batch_normalization_8
max_pooling2d_7
conv2d_6
batch_normalization_9
max_pooling2d_8
conv2d_7
batch_normalization_10
max_pooling2d_9
conv2d_8
batch_normalization_11
max_pooling2d_10
flatten_1
dropout_1
dense_2
batch_normalization_12
dense_3
```

```
In [29]: from tensorflow.keras.models import Model
```

```
feature_extractor = Model(
    inputs=cnn.input,
    outputs=cnn.get_layer("conv2d_7").output
)
```

```
print(" Feature extractor created from conv2d_7")
```

Feature extractor created from conv2d_7

```
In [30]: X_train_features = feature_extractor.predict(X_train)
         X_test_features  = feature_extractor.predict(X_test)

         print("Before flatten:", X_train_features.shape)
```

21/21 ————— 26s 1s/step

6/6 ————— 8s 1s/step

Before flatten: (646, 4, 4, 4)

```
In [37]: feature_extractor.save("FINAL_FEATURE_EXTRACTOR.h5")
print(" Feature extractor model saved successfully")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Feature extractor model saved successfully

```
In [31]: X_train_features = X_train_features.reshape(X_train_features.shape[0], -1)
X_test_features = X_test_features.reshape(X_test_features.shape[0], -1)

print("After flatten:", X_train_features.shape)
```

After flatten: (646, 64)

```
In [32]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam

fcnn = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_features.shape[1],)),
    BatchNormalization(),
    Dropout(0.25),

    Dense(64, activation='relu'),
    BatchNormalization(),

    Dense(2, activation='softmax')
])

fcnn.compile(
    optimizer=Adam(1e-4),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

fcnn.summary()
```

C:\Users\Pritam\miniconda3\envs\data_analysis_env\lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	8,320
batch_normalization_13 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 64)	8,256
batch_normalization_14 (BatchNormalization)	(None, 64)	256
dense_6 (Dense)	(None, 2)	130

Total params: 17,474 (68.26 KB)

Trainable params: 17,090 (66.76 KB)

Non-trainable params: 384 (1.50 KB)

```
In [34]: fcnn.compile(  
         optimizer=Adam(1e-4),  
         loss='categorical_crossentropy',  
         metrics=['accuracy']  
         )
```

```
In [36]: fcnn_history = fcnn.fit(  
         X_train_features, y_train,  
         validation_data=(X_test_features, y_test),  
         epochs=20,  
         batch_size=32  
         )
```

```

Epoch 1/20
21/21 ————— 0s 10ms/step - accuracy: 0.8406 - loss: 0.3748 - val_accuracy: 0.8025 - val_loss: 0.4257
Epoch 2/20
21/21 ————— 0s 7ms/step - accuracy: 0.8359 - loss: 0.3673 - val_accuracy: 0.8025 - val_loss: 0.4208
Epoch 3/20
21/21 ————— 0s 8ms/step - accuracy: 0.8607 - loss: 0.3382 - val_accuracy: 0.8086 - val_loss: 0.4192
Epoch 4/20
21/21 ————— 0s 6ms/step - accuracy: 0.8498 - loss: 0.3641 - val_accuracy: 0.8148 - val_loss: 0.4177
Epoch 5/20
21/21 ————— 0s 7ms/step - accuracy: 0.8483 - loss: 0.3494 - val_accuracy: 0.7963 - val_loss: 0.4188
Epoch 6/20
21/21 ————— 0s 7ms/step - accuracy: 0.8576 - loss: 0.3413 - val_accuracy: 0.8210 - val_loss: 0.4150
Epoch 7/20
21/21 ————— 0s 7ms/step - accuracy: 0.8514 - loss: 0.3505 - val_accuracy: 0.8210 - val_loss: 0.4126
Epoch 8/20
21/21 ————— 0s 7ms/step - accuracy: 0.8467 - loss: 0.3618 - val_accuracy: 0.8210 - val_loss: 0.4093
Epoch 9/20
21/21 ————— 0s 8ms/step - accuracy: 0.8653 - loss: 0.3366 - val_accuracy: 0.8210 - val_loss: 0.4070
Epoch 10/20
21/21 ————— 0s 8ms/step - accuracy: 0.8545 - loss: 0.3341 - val_accuracy: 0.8210 - val_loss: 0.4060
Epoch 11/20
21/21 ————— 0s 9ms/step - accuracy: 0.8483 - loss: 0.3241 - val_accuracy: 0.8272 - val_loss: 0.4040
Epoch 12/20
21/21 ————— 0s 7ms/step - accuracy: 0.8622 - loss: 0.3226 - val_accuracy: 0.8272 - val_loss: 0.3986
Epoch 13/20
21/21 ————— 0s 7ms/step - accuracy: 0.8793 - loss: 0.3221 - val_accuracy: 0.8272 - val_loss: 0.3959
Epoch 14/20
21/21 ————— 0s 7ms/step - accuracy: 0.8545 - loss: 0.3310 - val_accuracy: 0.8333 - val_loss: 0.3941
Epoch 15/20
21/21 ————— 0s 6ms/step - accuracy: 0.8700 - loss: 0.3138 - val_accuracy: 0.8272 - val_loss: 0.3951
Epoch 16/20
21/21 ————— 0s 7ms/step - accuracy: 0.8777 - loss: 0.3084 - val_accuracy: 0.8210 - val_loss: 0.3964
Epoch 17/20
21/21 ————— 0s 7ms/step - accuracy: 0.8746 - loss: 0.3128 - val_accuracy: 0.8333 - val_loss: 0.3929
Epoch 18/20
21/21 ————— 0s 7ms/step - accuracy: 0.8824 - loss: 0.2827 - val_accuracy: 0.8457 - val_loss: 0.3909
Epoch 19/20
21/21 ————— 0s 7ms/step - accuracy: 0.8715 - loss: 0.2963 - val_accuracy: 0.8457 - val_loss: 0.3919
Epoch 20/20
21/21 ————— 0s 7ms/step - accuracy: 0.8731 - loss: 0.3143 - val_accuracy: 0.8457 - val_loss: 0.3905

```

```
In [38]: fcnn.save("FINAL_FCNN_PARKINSON.h5")
         print(" FCNN model saved successfully")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

FCNN model saved successfully

```
In [40]: import os

         files = ["CUSTOM_CNN_PARKINSON.h5",
                  "FINAL_FEATURE_EXTRACTOR.h5",
```

```
"FINAL_FCNN_PARKINSON.h5"]
```

```
for f in files:  
    print(f, "→", "FOUND " if os.path.exists(f) else "NOT FOUND ")
```

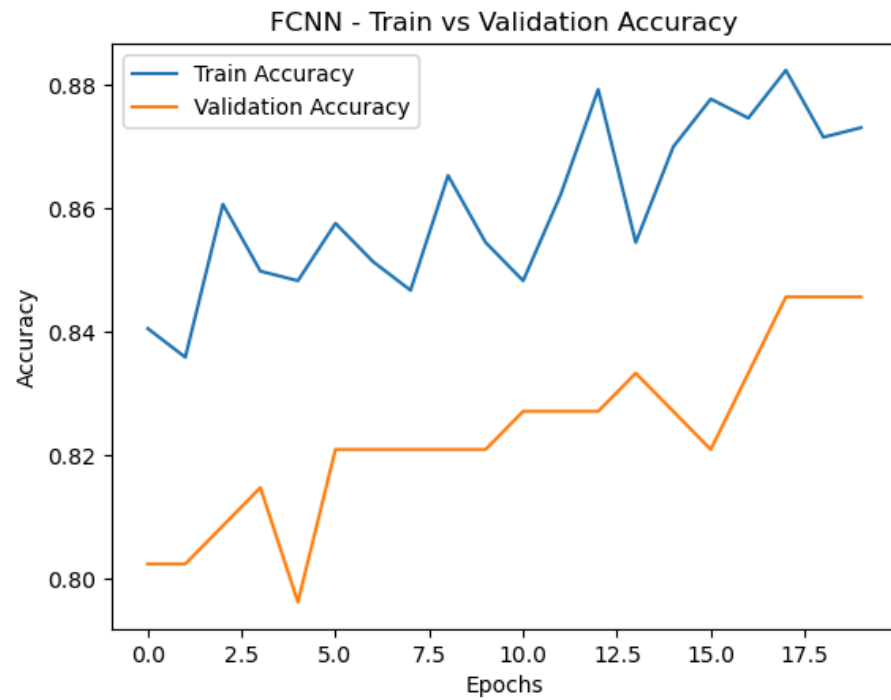
CUSTOM_CNN_PARKINSON.h5 → FOUND

FINAL_FEATURE_EXTRACTOR.h5 → FOUND

FINAL_FCNN_PARKINSON.h5 → FOUND

In [41]: `import matplotlib.pyplot as plt`

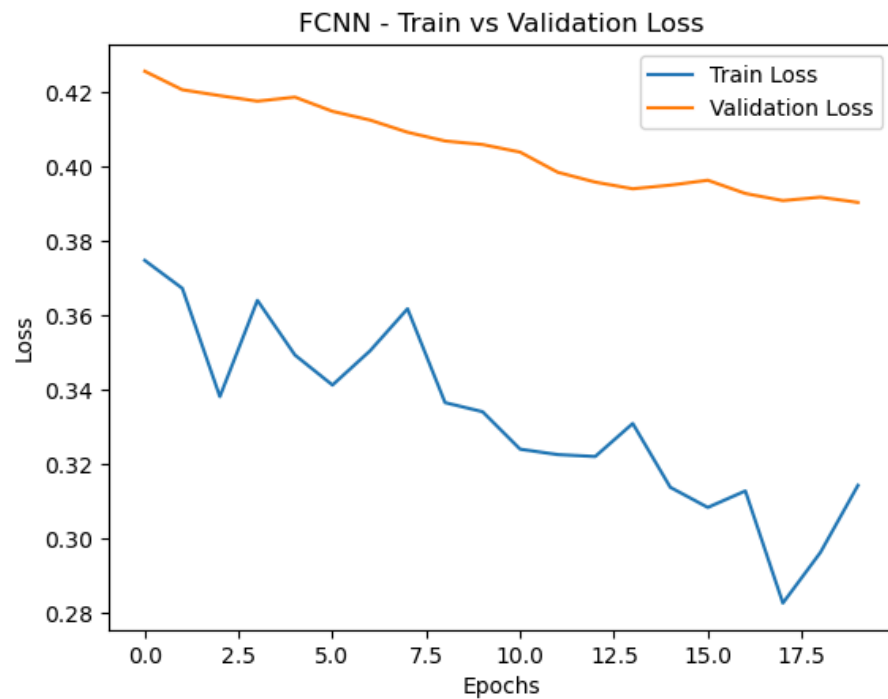
```
plt.figure()  
plt.plot(fcnn_history.history['accuracy'], label='Train Accuracy')  
plt.plot(fcnn_history.history['val_accuracy'], label='Validation Accuracy')  
  
plt.title('FCNN - Train vs Validation Accuracy')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```



In [42]: `#FCNN LOSS CURVE`

```
In [43]: plt.figure()
plt.plot(fcnn_history.history['loss'], label='Train Loss')
plt.plot(fcnn_history.history['val_loss'], label='Validation Loss')

plt.title('FCNN - Train vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
In [44]: #EVOLUTION
```

```
In [45]: from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

y_pred = fcnn.predict(X_test_features)

y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

cm = confusion_matrix(y_true, y_pred_classes)
```

```
acc = accuracy_score(y_true, y_pred_classes)

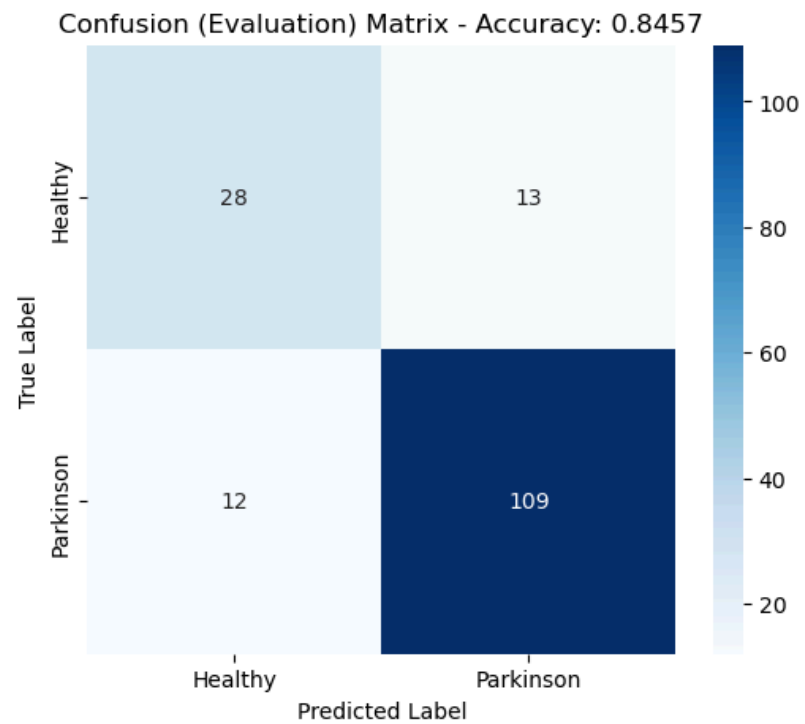
print(" Accuracy:", acc)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Healthy", "Parkinson"],
            yticklabels=["Healthy", "Parkinson"])

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title(f"Confusion (Evaluation) Matrix - Accuracy: {acc:.4f}")
plt.show()
```

6/6 ————— 0s 22ms/step

Accuracy: 0.845679012345679



In []: